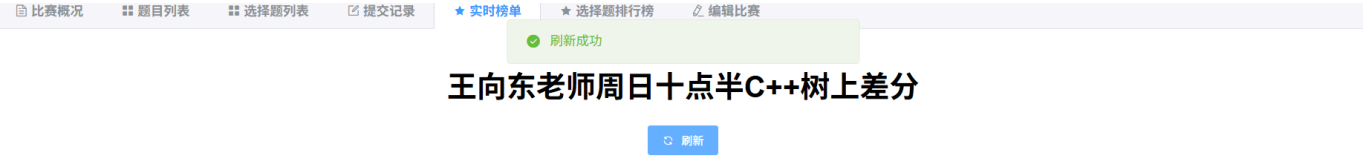
线段树

人员

于子珈、周子一、陈洛冉、赵熙羽、牛同泽、秦显森、谢亚锴 到课

上周作业检查

上周作业链接: https://cppoj.kids123code.com/contest/3588



王向东老师周日十点半C++树上差分

刷新

#	用户名	姓名	编程分	时间	A	B	C	D
1	yuzijia1	于子珈	400	40188	100	100	100	100
2	qinxiansen	秦显森	400	48058	100	100	100	100
3	zhouziyi	周子一	300	49795	100	100	100	
4	zhaoxiyu	赵熙羽	300	59765	100	100	100	
5	yangjinshuo	杨谨硕	201	2648	100	100	1	
6	lizihan	李子瀚	200	4253	100	100		
7	xieyakai	谢亚锴	200	6243	100	100		
8	niutongze	牛同泽	200	6626	100	100		
9	chenluoran	陈洛冉	200	8315	100	100		
10	siyunxin	司云心	100	7625	100			

本周作业

https://cppoj.kids123code.com/contest/3731 (课上讲了 A ~ B 题, 课后作业是 A B 题必做, C 题选做)

课堂表现

今天讲了 线段树 这个知识点, 主要讲了 单点修改、区间查询 的操作

同学们课上吸收的都不错, 但 线段树 比较难的是写代码, 代码会相对复杂一些, 同学们课下要把这几道线段树的题多写几遍, 增加熟练度。

课堂内容

[JLOI2014] 松鼠的新家 (上周作业)

树上差分板题

```
#include <bits/stdc++.h>

using namespace std;

const int N = 3e5 + 5, M = 20;
int c[N];
```

```
vector<int> vec[N];
int dis[N], f[N][M], w[N];

void dfs(int u, int fa) {
  dis[u] = dis[fa]+1, f[u][0] = fa;
  for (int i = 1; i < M; ++i) f[u][i] = f[f[u][i-1]][i-1];

  for (int i : vec[u]) {
    if (i != fa) dfs(i, u);
  }
}

int lca(int a, int b) {
  if (dis[a] < dis[b]) return lca(b, a);

  for (int i = M-1; i >= 0; --i) {
    if (dis[f[a][i]] >= dis[b]) a = f[a][i];
  }

  if (a == b) return a;

  for (int i = M-1; i >= 0; --i) {
    if (f[a][i] != f[b][i]) a = f[a][i], b = f[b][i];
  }

  return f[a][0];
}

void dfs2(int u, int fa) {
  for (int i : vec[u]) {
    if (i == fa) continue;
    dfs2(i, u);
    w[u] += w[i];
  }
}

int main()
{
  int n; cin >> n;
  for (int i = 1; i <= n; ++i) cin >> c[i];
  for (int i = 1; i <= n-1; ++i) {
    int a, b; cin >> a >> b;
    vec[a].push_back(b), vec[b].push_back(a);
  }

  dfs(1, 0);

  for (int i = 2; i <= n; ++i) {
    int a = c[i-1], b = c[i];
    int u = lca(a, b), fu = f[u][0];
    ++w[a], ++w[b], --w[u], --w[fu];
  }

  dfs2(1, 0);
```

```
  for (int i = 2; i <= n; ++i) w[c[i]]--;
  for (int i = 1; i <= n; ++i) cout << w[i] << endl;
  return 0;
}
```

I Hate It

线段树模板题, 单点修改, 区间查询最大值

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 2e5 + 5;
int w[maxn];
struct node {
    int l, r, maxx;
} tr[maxn*4];

void pushup(int u) { tr[u].maxx = max(tr[u*2].maxx, tr[u*2+1].maxx); }
void build(int u, int l, int r) {
    tr[u] = {l, r, 0};
    if (l == r) { tr[u].maxx = w[l]; return; }

    int mid = (l + r) / 2;
    build(u*2, l, mid), build(u*2+1, mid+1, r);
    pushup(u);
}
void modify(int u, int pos, int k) {
    if (tr[u].l == tr[u].r) { tr[u].maxx = max(tr[u].maxx, k); return; }

    int mid = (tr[u].l + tr[u].r) / 2;
    if (pos <= mid) modify(u*2, pos, k);
    else modify(u*2+1, pos, k);
    pushup(u);
}
int query(int u, int l, int r) {
    if (tr[u].l>=l && tr[u].r<=r) return tr[u].maxx;

    int mid = (tr[u].l + tr[u].r) / 2;
    int res = 0;
    if (l <= mid) res = query(u*2, l, r);
    if (r > mid) res = max(res, query(u*2+1, l, r));
    return res;
}

int main()
{
    int n, m; cin >> n >> m;
    for (int i = 1; i <= n; ++i) cin >> w[i];
```

```
    build(1, 1, n);

    while (m -- ) {
        char op[2]; cin >> op;
        if (op[0] == 'Q') {
            int l, r; cin >> l >> r;
            cout << query(1, l, r) << endl;
        } else {
            int pos, k; cin >> pos >> k;
            modify(1, pos, k);
        }
    }
    return 0;
}
```

统计和

线段树模板题, 单点修改, 区间查询区间和

```
#include <bits/stdc++.h>

using namespace std;

typedef long long LL;
const int maxn = 1e5 + 5;
struct node {
  int l, r;
  LL sum;
} tr[maxn*4];

void pushup(int u) { tr[u].sum = tr[u*2].sum + tr[u*2+1].sum; }

void build(int u, int l, int r) {
  tr[u] = {l, r, 0};
  if (l == r) return;
  int mid = (l + r) / 2;
  build(u*2, l, mid), build(u*2+1, mid+1, r);
}

void modify(int u, int pos, int k) {
  if (tr[u].l==pos && tr[u].r==pos) {
    tr[u].sum += k; return;
  }

  int mid = (tr[u].l + tr[u].r) / 2;
  if (pos <= mid) modify(u*2, pos, k);
  else modify(u*2+1, pos, k);
  pushup(u);
}

LL query(int u, int l, int r) {
```

```
  if (tr[u].l>=l && tr[u].r<=r) return tr[u].sum;

  LL res = 0;
  int mid = (tr[u].l + tr[u].r) / 2;
  if (l <= mid) res += query(u*2, l, r);
  if (r > mid) res += query(u*2+1, l, r);
  return res;
}

int main()
{
  int n, m; cin >> n >> m;
  build(1, 1, n);
  while (m -- ) {
    char op[2]; cin >> op;
    if (op[0] == 'x') {
      int a, b; cin >> a >> b; modify(1, a, b);
    } else {
      int l, r; cin >> l >> r;
//      cout << "------------------- ";
      cout << query(1, l, r) << "\n";
    }
  }
  return 0;
}
```

[JSOI2008] 最大数

本质上也是单点修改, 区间查询最大值

```
#include <bits/stdc++.h>

using namespace std;

typedef long long LL;
const int maxn = 2e5 + 5;
const LL inf = 0x3f3f3f3f3f3f3f3f;
struct node {
  int l, r;
  LL maxx;
} tr[maxn*4];

void pushup(int u) {
  tr[u].maxx = max(tr[u*2].maxx, tr[u*2+1].maxx);
}

void build(int u, int l, int r) {
  tr[u] = {l, r, -inf};
  if (l == r) return;
  int mid = (l + r) / 2;
  build(u*2, l, mid), build(u*2+1, mid+1, r);
```

```
}

void modify(int u, int pos, LL k) {
  if (tr[u].l==pos && tr[u].r==pos) {
    tr[u].maxx = k; return;
  }

  int mid = (tr[u].l+tr[u].r) / 2;
  if (pos <= mid) modify(u*2, pos, k);
  else modify(u*2+1, pos, k);
  pushup(u);
}

LL query(int u, int l, int r) {
  if (tr[u].l>=l && tr[u].r<=r) return tr[u].maxx;

  int mid = (tr[u].l+tr[u].r) / 2;
  LL res = -inf;
  if (l <= mid) res = query(u*2, l, r);
  if (r > mid) res = max(res, query(u*2+1, l, r));
  return res;
}

int main()
{
  int n; LL mod; cin >> n >> mod;
  build(1, 1, n);
  LL t = 0;
  for (int i = 1, id = 0; i <= n; ++i) {
    char op[2]; cin >> op;
    if (op[0] == 'A') {
      LL n; cin >> n;
      n = (n + t) % mod;
      ++id;
      modify(1, id, n);
    } else {
      int k; cin >> k;
      t = query(1, id-k+1, id);

      cout << t << endl;
    }
  }
  return 0;
}
```